



دانشگاه هرمزگان
دانشکده فنی و مهندسی
گروه مهندسی برق و کامپیوتر

داده‌ساختارها و الگوریتم‌ها

جلسه ششم – تقسیم و حل: مرتب‌سازی ادغامی و تحلیل الگوریتم‌های بازگشتی

دکتر شهرام گلزاری

پاییز ۱۴۰۵

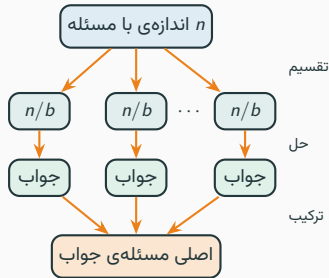
در پایان این جلسه انتظار داریم که:

- ایده‌ی **مرتب‌سازی ادغامی** را بشناسیم و اجرای آن را روی یک آرایه دنبال کنیم.
- رویه‌ی MERGE را خطبه‌خط بفهمیم و ببینیم چرا زمانش $\Theta(n)$ است.
- برای یک الگوریتم تقسیم و حل **رابطه‌ی بازگشتی** بنویسیم.
- با **درخت بازگشت** نشان دهیم زمان مرتب‌سازی ادغامی $\Theta(n \lg n)$ است.

یادآوری: روش تقسیم و حل

«بسیاری از الگوریتم‌های سودمند ساختاری **بازگشتی** دارند: برای حل یک مسئله‌ی داده‌شده، یک یا چند بار **بازگشت می‌کنند** (خودشان را صدا می‌زنند) تا زیرمسئله‌هایی را که ارتباط نزدیکی با آن دارند حل کنند. این الگوریتم‌ها معمولاً از روش **تقسیم و حل** پیروی می‌کنند: مسئله را به چند زیرمسئله می‌شکنند که شبیه مسئله‌ی اصلی‌اند اما اندازه‌ی کوچک‌تری دارند، زیرمسئله‌ها را به‌صورت بازگشتی حل می‌کنند و سپس این جواب‌ها را ترکیب می‌کنند تا جوابی برای مسئله‌ی اصلی بسازند.»

CLRS, 4th ed., صفحه ۳۴ (ترجمه ماشینی)



«در روش تقسیم و حل، اگر مسئله به اندازه‌ی کافی کوچک باشد (حالت پایه)، آن را مستقیماً و بدون بازگشت حل می‌کنید. در غیر این صورت (حالت بازگشتی)، سه گام مشخص را انجام می‌دهید:

- **تقسیم مسئله** به یک یا چند زیرمسئله که نمونه‌های کوچک‌تری از همان مسئله‌اند.
- **حل** زیرمسئله‌ها با حل بازگشتی آن‌ها.
- **ترکیب** جواب‌های زیرمسئله‌ها برای ساختن جوابی برای مسئله‌ی اصلی.»

CLRS, 4th ed., صفحه ۳۴ (ترجمه ماشینی).

ایده‌ی مرتب‌سازی ادغامی

«الگوریتم مرتب‌سازی ادغامی از نزدیک از روش تقسیم و حل پیروی می‌کند. در هر گام، یک زیرآرایه‌ی $A[p : r]$ را مرتب می‌کند؛ از کل آرایه‌ی $A[1 : n]$ شروع می‌کند و با بازگشت به زیرآرایه‌های کوچک‌تر و کوچک‌تر می‌رسد. مرتب‌سازی ادغامی این‌گونه کار می‌کند:

- **تقسیم** زیرآرایه‌ی $A[p : r]$ که باید مرتب شود به دو زیرآرایه‌ی مجاور، هر کدام به اندازه‌ی نصف. برای این کار، نقطه‌ی میانی q از $A[p : r]$ را حساب کنید (با گرفتن میانگین p و r) و $A[p : r]$ را به زیرآرایه‌های $A[p : q]$ و $A[q + 1 : r]$ تقسیم کنید.
- **حل** با مرتب کردن هر یک از دو زیرآرایه‌ی $A[p : q]$ و $A[q + 1 : r]$ به‌صورت بازگشتی با مرتب‌سازی ادغامی.
- **ترکیب** با ادغام دو زیرآرایه‌ی مرتب $A[p : q]$ و $A[q + 1 : r]$ و بازگرداندن آن‌ها به $A[p : r]$ ، که جواب مرتب‌شده را تولید می‌کند.»

CLRS, 4th ed.، صفحه ۳۴ و ۳۵ (ترجمه ماشینی)

«بازگشت وقتی "به ته می‌رسد" (یعنی به حالت پایه می‌رسد) که زیرآرایه‌ی $A[p : r]$ که باید مرتب شود فقط ۱ عنصر داشته باشد، یعنی وقتی p برابر r باشد. (...) زیرآرایه‌ای که فقط یک عنصر دارد همیشه مرتب است.»

«عمل کلیدی الگوریتم مرتب‌سازی ادغامی در گام "ترکیب" رخ می‌دهد که دو زیرآرایه‌ی مجاور و مرتب را ادغام می‌کند. عمل ادغام را رویه‌ی کمکی $\text{MERGE}(A, p, q, r)$ در صفحه‌ی بعد انجام می‌دهد، که در آن A یک آرایه است و p, q و r اندیس‌هایی در آرایه هستند به‌طوری که $p \leq q < r$. این رویه فرض می‌کند زیرآرایه‌های مجاور $A[p : q]$ و $A[q + 1 : r]$ قبلاً به‌صورت بازگشتی مرتب شده‌اند. این رویه دو زیرآرایه‌ی مرتب را ادغام می‌کند تا یک زیرآرایه‌ی مرتب واحد بسازد که جایگزین زیرآرایه‌ی فعلی $A[p : r]$ می‌شود.»

CLRS, 4th ed.، صفحه ۳۵ (ترجمه ماشینی)

رویهی MERGE

«رویه‌ی MERGE با جزئیات این‌گونه کار می‌کند. دو زیرآرایه‌ی $A[p : q]$ و $A[q + 1 : r]$ را در آرایه‌های موقت L و R (“چپ” و “راست”) کپی می‌کند و سپس مقدارهای L و R را ادغام می‌کند و به $A[p : r]$ برمی‌گرداند.»

CLRS, 4th ed.، صفحه ۳۵ (ترجمه ماشینی)

```

MERGE( $A, p, q, r$ )
1   $n_L = q - p + 1$            // length of  $A[p : q]$ 
2   $n_R = r - q$                // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$        // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$        // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                        //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                        //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                        //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //   copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19          $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //   remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 

```

خط‌های ۱ تا ۳

«خط‌های ۱ و ۲ طول‌های n_L و n_R زیرآرایه‌های $A[p : q]$ و $A[q + 1 : r]$ را به ترتیب حساب می‌کنند. سپس خط ۳ آرایه‌های $L[0 : n_L - 1]$ و $R[0 : n_R - 1]$ را به ترتیب با طول‌های n_L و n_R می‌سازد.»
CLRS, 4th ed., صفحه ۳۵ و ۳۶ (ترجمه ماشینی)

```

MERGE( $A, p, q, r$ )
1   $n_L = q - p + 1$            // length of  $A[p : q]$ 
2   $n_R = r - q$                // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$        // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$        // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                        //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                        //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                        //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 //   copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19          $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 //   remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 

```

خط‌های ۴ تا ۷

«حلقه‌ی for خط‌های ۴-۵ زیرآرایه‌ی $A[p : q]$ را در L کپی می‌کند و حلقه‌ی for خط‌های ۶-۷ زیرآرایه‌ی $A[q + 1 : r]$ را در R کپی می‌کند.»

(ترجمه ماشینی) *CLRS*, 4th ed. صفحه ۳۶

```

MERGE( $A, p, q, r$ )
1   $n_L = q - p + 1$            // length of  $A[p : q]$ 
2   $n_R = r - q$                // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$        // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$        // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                        //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                        //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                        //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
    // copy the smallest unmerged element back into  $A[p : r]$ .
12 while  $i < n_L$  and  $j < n_R$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
18          $k = k + 1$ 
19 // Having gone through one of  $L$  and  $R$  entirely, copy the
    // remainder of the other to the end of  $A[p : r]$ .
20 while  $i < n_L$ 
21      $A[k] = L[i]$ 
22      $i = i + 1$ 
23      $k = k + 1$ 
24 while  $j < n_R$ 
25      $A[k] = R[j]$ 
26      $j = j + 1$ 
27      $k = k + 1$ 
    
```

خط‌های ۸ تا ۱۸

«خط‌های ۸-۱۸، که در شکل ۳.۲ نشان داده شده‌اند، گام‌های پایه را انجام می‌دهند. حلقه‌ی `while` خط‌های ۱۲-۱۸ بارها کوچک‌ترین مقداری از L و R را که هنوز به $A[p : r]$ برگردانده نشده است پیدا می‌کند و آن را به آن‌جا برمی‌گرداند. همان‌طور که توضیحات نشان می‌دهند، اندیس k موقعیتی از A را می‌دهد که در حال پر شدن است و اندیس‌های i و j به ترتیب موقعیت کوچک‌ترین مقدارهای باقی‌مانده در L و R را می‌دهند.»

(ترجمه ماشینی) *CLRS, 4th ed.*، صفحه ۳۶ و ۳۸

```

MERGE( $A, p, q, r$ )
1   $n_L = q - p + 1$            // length of  $A[p : q]$ 
2   $n_R = r - q$                // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$        // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$        // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                        //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                        //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                        //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
    // copy the smallest unmerged element back into  $A[p : r]$ .
12 while  $i < n_L$  and  $j < n_R$ 
13     if  $L[i] \leq R[j]$ 
14          $A[k] = L[i]$ 
15          $i = i + 1$ 
16     else  $A[k] = R[j]$ 
17          $j = j + 1$ 
18      $k = k + 1$ 
19 // Having gone through one of  $L$  and  $R$  entirely, copy the
    // remainder of the other to the end of  $A[p : r]$ .
20 while  $i < n_L$ 
21      $A[k] = L[i]$ 
22      $i = i + 1$ 
23      $k = k + 1$ 
24 while  $j < n_R$ 
25      $A[k] = R[j]$ 
26      $j = j + 1$ 
27      $k = k + 1$ 
    
```

خط‌های ۲۰ تا ۲۷

«سرانجام، یا همه‌ی L یا همه‌ی R به $A[p : r]$ برگردانده می‌شود و این حلقه پایان می‌یابد. اگر حلقه به این دلیل پایان یابد که همه‌ی R برگردانده شده است، یعنی چون j برابر n_R است، آنگاه i هنوز کمتر از n_L است، پس بخشی از L هنوز باید برگردانده شود و این مقادارها بزرگ‌ترین مقادارها در هر دوی L و R هستند. در این حالت، حلقه‌ی **while** خط‌های ۲۰-۲۳ این مقادارهای باقی‌مانده‌ی L را در چند موقعیت آخر $A[p : r]$ کپی می‌کند. چون j برابر n_R است، حلقه‌ی **while** خط‌های ۲۴-۲۷ صفر بار تکرار می‌شود. اگر در عوض حلقه‌ی **while** خط‌های ۱۲-۱۸ به این دلیل پایان یابد که i برابر n_L است، آنگاه همه‌ی L قبلاً به $A[p : r]$ برگردانده شده است و حلقه‌ی **while** خط‌های ۲۴-۲۷ مقادارهای باقی‌مانده‌ی R را به انتهای $A[p : r]$ برمی‌گرداند.»

(ترجمه ماشینی) *CLRS*, 4th ed., صفحه ۳۸

```

MERGE(A, p, q, r)
1   $n_L = q - p + 1$            // length of  $A[p : q]$ 
2   $n_R = r - q$                // length of  $A[q + 1 : r]$ 
3  let  $L[0 : n_L - 1]$  and  $R[0 : n_R - 1]$  be new arrays
4  for  $i = 0$  to  $n_L - 1$        // copy  $A[p : q]$  into  $L[0 : n_L - 1]$ 
5       $L[i] = A[p + i]$ 
6  for  $j = 0$  to  $n_R - 1$        // copy  $A[q + 1 : r]$  into  $R[0 : n_R - 1]$ 
7       $R[j] = A[q + j + 1]$ 
8   $i = 0$                      //  $i$  indexes the smallest remaining element in  $L$ 
9   $j = 0$                      //  $j$  indexes the smallest remaining element in  $R$ 
10  $k = p$                      //  $k$  indexes the location in  $A$  to fill
11 // As long as each of the arrays  $L$  and  $R$  contains an unmerged element,
12 // copy the smallest unmerged element back into  $A[p : r]$ .
13 while  $i < n_L$  and  $j < n_R$ 
14     if  $L[i] \leq R[j]$ 
15          $A[k] = L[i]$ 
16          $i = i + 1$ 
17     else  $A[k] = R[j]$ 
18          $j = j + 1$ 
19          $k = k + 1$ 
20 // Having gone through one of  $L$  and  $R$  entirely, copy the
21 // remainder of the other to the end of  $A[p : r]$ .
22 while  $i < n_L$ 
23      $A[k] = L[i]$ 
24      $i = i + 1$ 
25      $k = k + 1$ 
26 while  $j < n_R$ 
27      $A[k] = R[j]$ 
28      $j = j + 1$ 
29      $k = k + 1$ 

```

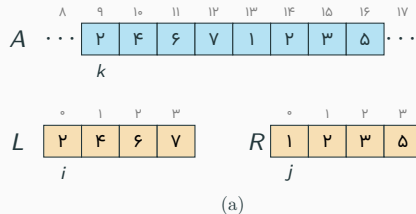
«عملکرد حلقه‌ی `while` در خط‌های ۸–۱۸ در فراخوانی $\text{MERGE}(A, 9, 12, 16)$ ، وقتی زیرآرایه‌ی $A[9 : 16]$ شامل مقدارهای $\langle 2, 4, 6, 7, 1, 2, 3, 5 \rangle$ است. پس از تخصیص آرایه‌های L و R و کپی در آن‌ها، آرایه‌ی L شامل $\langle 2, 4, 6, 7 \rangle$ و آرایه‌ی R شامل $\langle 1, 2, 3, 5 \rangle$ است. خانه‌های **کرم** در A مقدارهای نهایی‌شان را دارند و خانه‌های **کرم** در L و R مقدارهایی را دارند که هنوز باید به A برگردانده شوند. روی هم، خانه‌های کرم همیشه همان مقدارهایی را تشکیل می‌دهند که در ابتدا در $A[9 : 16]$ بودند. خانه‌های **آبی** در A مقدارهایی را دارند که رونویسی خواهند شد و خانه‌های **تیره** در L و R مقدارهایی را دارند که قبلاً به A برگردانده شده‌اند.»

CLRS, 4th ed.، صفحه ۳۷، زیرنویس شکل ۳.۲ (ترجمه ماشینی)

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18      $k = k + 1$ 
19   :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 

```



تیره آبی کرم

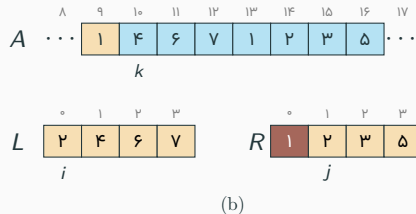
پیش از تکرار ۱: $L[0] = 2$ و $R[0] = 1$ مقایسه می‌شوند. چون $2 \leq 1$ برقرار نیست، else اجرا می‌شود: $A[9] = 1$ و j یک واحد جلو می‌رود.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.


```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18    $k = k + 1$ 
19   :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 

```



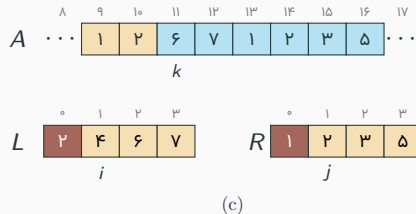
تیره آبی کرم

پیش از تکرار ۲: $L[0] = 2$ و $R[1] = 2$. شرط $2 \leq 2$ برقرار است، پس در حالت برابر از L برداشته می‌شود: $A[10] = 2$ و i جلو می‌رود.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18      $k = k + 1$ 
    :
    :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 
    
```



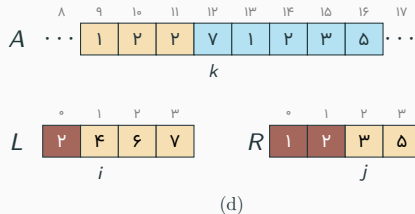
پیش از تکرار ۳: $L[1] = 4$ و $R[1] = 2$. عنصر R کوچک‌تر است: $A[11] = 2$.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18    $k = k + 1$ 
19   :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 

```



تیره آبی کرم

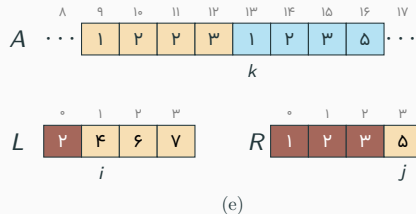
پیش از تکرار ۴: $L[1] = 4$ و $R[2] = 3$. باز هم از R : $A[12] = 3$.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18    $k = k + 1$ 
19   :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 

```

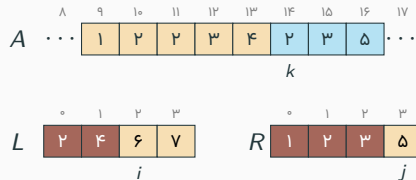


پیش از تکرار ۵: $L[1] = ۴$ و $R[۳] = ۵$. این بار از L : $A[۱۳] = ۴$.

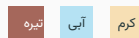
(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS, 4th ed.*

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18      $k = k + 1$ 
    :
    :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 
    
```



(f)



پیش از تکرار ۶: $L[2] = 6$ و $R[3] = 5$ از $A[14] = 5$.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS, 4th ed.*

12 while $i < n_L$ and $j < n_R$

13 if $L[i] \leq R[j]$

14 $A[k] = L[i]$

15 $i = i + 1$

16 else $A[k] = R[j]$

17 $j = j + 1$

18 $k = k + 1$

⋮

20 while $i < n_L$

21 $A[k] = L[i]$

22 $i = i + 1$

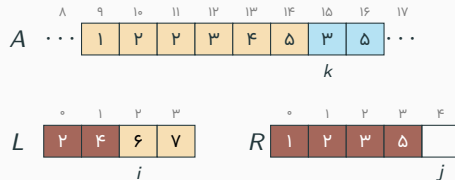
23 $k = k + 1$

24 while $j < n_R$

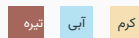
25 $A[k] = R[j]$

26 $j = j + 1$

27 $k = k + 1$



(g)



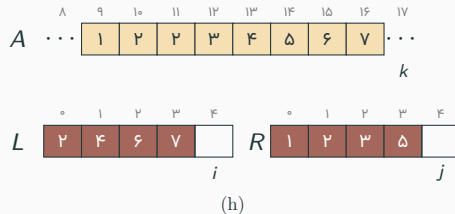
حالت (g): $n_R = 4$ ؛ همه‌ی R به A برگشته و حلقه‌ی خط‌های ۱۲-۱۸ تمام می‌شود. حلقه‌ی خط‌های ۲۰-۲۳ مقدارهای $L[2 : 3]$ یعنی ۶ و ۷ را در $A[15 : 16]$ کپی می‌کند.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

```

12 while  $i < n_L$  and  $j < n_R$ 
13   if  $L[i] \leq R[j]$ 
14      $A[k] = L[i]$ 
15      $i = i + 1$ 
16   else  $A[k] = R[j]$ 
17      $j = j + 1$ 
18    $k = k + 1$ 
19   :
20 while  $i < n_L$ 
21    $A[k] = L[i]$ 
22    $i = i + 1$ 
23    $k = k + 1$ 
24 while  $j < n_R$ 
25    $A[k] = R[j]$ 
26    $j = j + 1$ 
27    $k = k + 1$ 

```



تیره آبی کرم

حالت (h): پایان. حلقه‌ی خط‌های ۲۴-۲۷ صفر بار اجرا شد و زیرآرایه‌ی $A[9 : 16]$ مرتب است.

(بازترسیم شکل ۳.۲) صفحه ۳۷، *CLRS*, 4th ed.

«برای اینکه ببینیم رویه‌ی MERGE در زمان $\Theta(n)$ اجرا می‌شود، که در آن $n = r - p + 1$ ، توجه کنید که هر یک از خط‌های ۱-۳ و ۸-۱۰ زمان ثابتی می‌گیرد و حلقه‌های for خط‌های ۴-۷ زمان $\Theta(n_L + n_R) = \Theta(n)$ می‌گیرند. برای به حساب آوردن سه حلقه‌ی while خط‌های ۱۲-۱۸، ۲۰-۲۳ و ۲۴-۲۷، توجه کنید که هر تکرار این حلقه‌ها دقیقاً یک مقدار را از L یا R به A برمی‌گرداند و هر مقدار دقیقاً یک بار به A برگردانده می‌شود. بنابراین، این سه حلقه روی هم در کل n تکرار انجام می‌دهند. چون هر تکرار هر یک از این سه حلقه زمان ثابتی می‌گیرد، کل زمان صرف‌شده در این سه حلقه $\Theta(n)$ است.»

CLRS, 4th ed.، صفحه ۳۸ (ترجمه ماشینی)

رویهی MERGE-SORT

«اکنون می‌توانیم از رویه‌ی MERGE به‌عنوان زیرروال در الگوریتم مرتب‌سازی ادغامی استفاده کنیم. رویه‌ی $\text{MERGE-SORT}(A, p, r)$ در صفحه‌ی روبه‌رو عنصرهای زیرآرایه‌ی $A[p : r]$ را مرتب می‌کند. اگر p برابر r باشد، زیرآرایه فقط ۱ عنصر دارد و بنابراین از قبل مرتب است. در غیر این صورت، باید $p < r$ باشد و MERGE-SORT گام‌های تقسیم، حل و ترکیب را اجرا می‌کند. گام تقسیم فقط اندیس q را حساب می‌کند که $A[p : r]$ را به دو زیرآرایه‌ی مجاور افراز می‌کند: $A[p : q]$ شامل $\lceil n/2 \rceil$ عنصر، و $A[q + 1 : r]$ شامل $\lfloor n/2 \rfloor$ عنصر. فراخوانی اولیه‌ی $\text{MERGE-SORT}(A, 1, n)$ کل آرایه‌ی $A[1 : n]$ را مرتب می‌کند.»

CLRS, 4th ed.، صفحه ۳۸ (ترجمه ماشینی)

```

MERGE-SORT( $A, p, r$ )
1  if  $p \geq r$                                 // zero or one element?
2      return
3   $q = \lfloor (p + r)/2 \rfloor$                        // midpoint of  $A[p : r]$ 
4  MERGE-SORT( $A, p, q$ )                         // recursively sort  $A[p : q]$ 
5  MERGE-SORT( $A, q + 1, r$ )                     // recursively sort  $A[q + 1 : r]$ 
6  // Merge  $A[p : q]$  and  $A[q + 1 : r]$  into  $A[p : r]$ .
7  MERGE( $A, p, q, r$ )

```

۳۹، CLRS، صفحه

• تقسیم: خط ۳ حل: خط‌های ۴ و ۵ ترکیب: خط ۷

۱	۲	۳	۴	۵	۶	۷	۸
۱۲	۳	۷	۹	۱۴	۶	۱۱	۲

تقسیم

تقسیم

تقسیم

جاری فراخوانی

MERGE-SORT($A, 1, 8$)

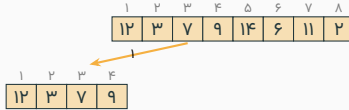
ادغام

ادغام

ادغام

۴۰ صفحه، *CLRS, 4th ed.*؛ برچسب‌های p, q و r حذف شده‌اند) (بازترسیم شکل ۴.۲؛

تقسیم



تقسیم

تقسیم

جاری فراخوانی

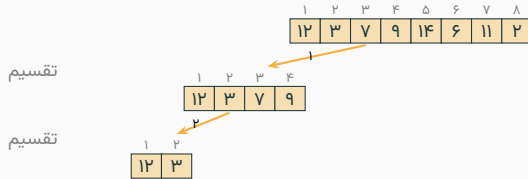
MERGE-SORT($A, 1, 4$)

ادغام

ادغام

ادغام

CLRS, 4th ed.، صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند)



جاری فراخوانی

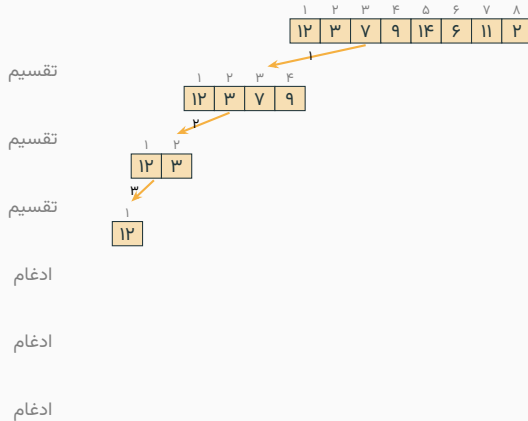
MERGE-SORT($A, 1, 2$)

ادغام

ادغام

ادغام

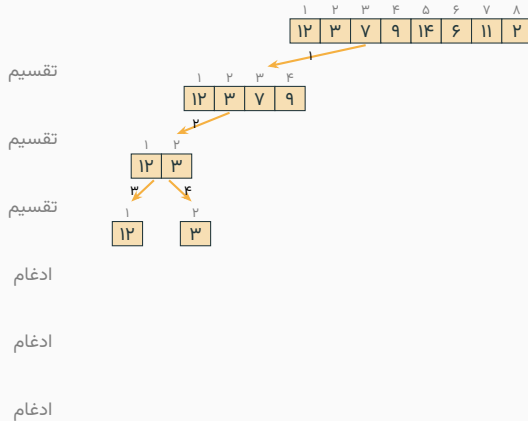
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*، صفحه ۴۰



جاری فراخوانی

MERGE-SORT($A, 1, 1$)

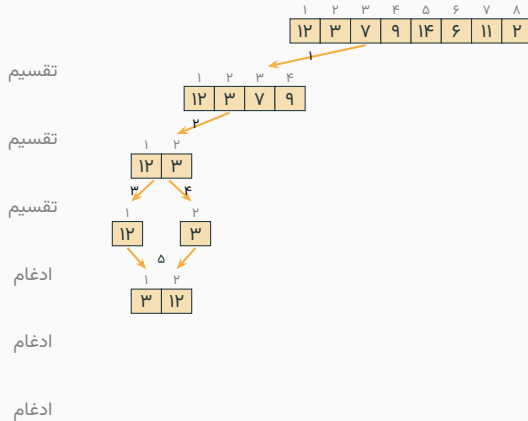
۴۰ صفحه ۴۰۲؛ برچسب‌های p ، q و r حذف شده‌اند) (بازترسیم شکل ۴.۲؛ *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT($A, 2, 2$)

(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*، صفحه ۴۰



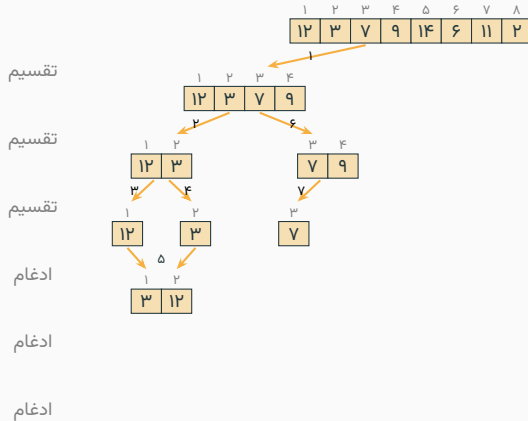
جاری فراخوانی

MERGE(A, 1, 1, 2)

(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*

MERGE-SORT($A, 3, 4$)

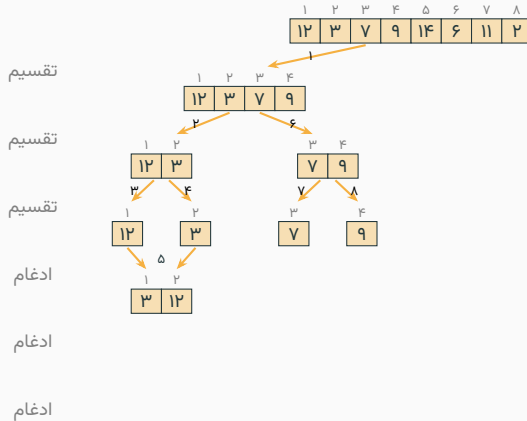
صفحه ۴۰، *CLRS, 4th ed.* (بازترسیم شکل ۴.۲؛ برجسب‌های p ، q و r حذف شده‌اند)



جاری فراخوانی

MERGE-SORT($A, 3, 3$)

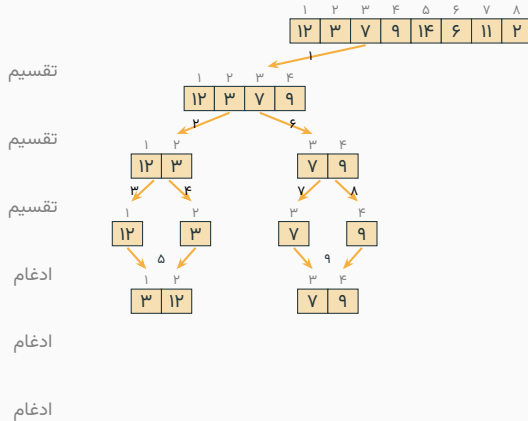
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*، صفحه ۴۰



جاری فراخوانی

MERGE-SORT($A, 4, 4$)

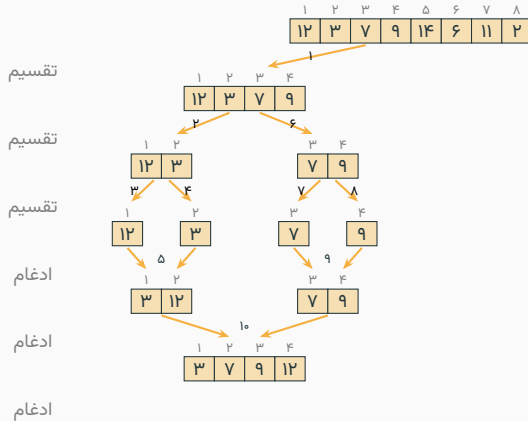
CLRS, 4th ed. صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند)



جاری فراخوانی

MERGE(A, 3, 3, 4)

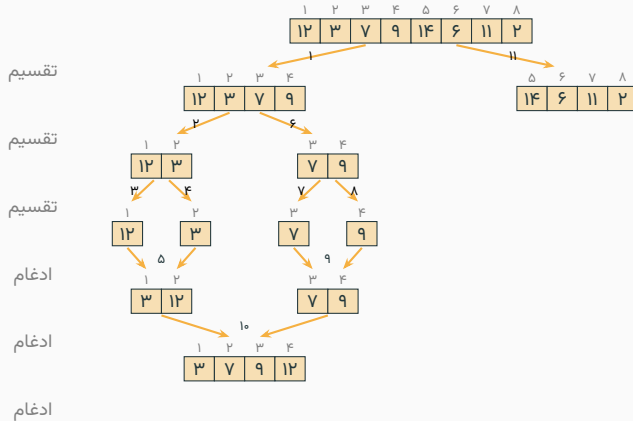
CLRS, 4th ed. صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند)



جاری فراخوانی

MERGE(A, 1, 2, 4)

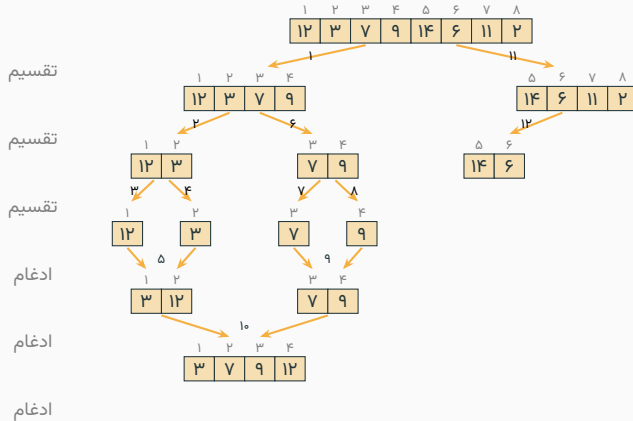
صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT($A, 5, 8$)

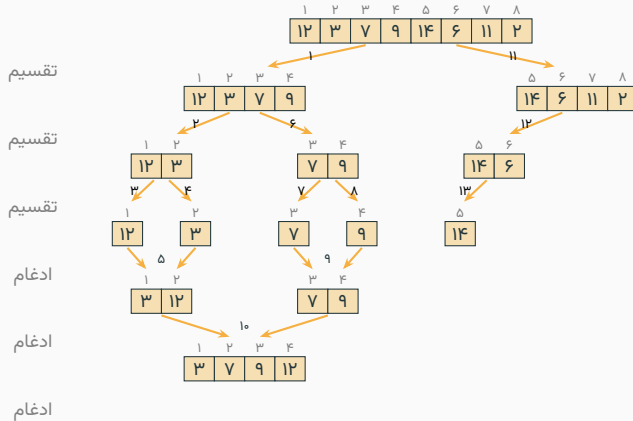
صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT($A, 5, 6$)

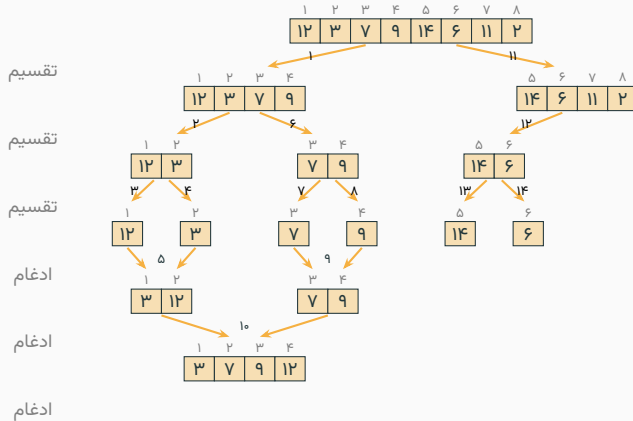
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT($A, 5, 5$)

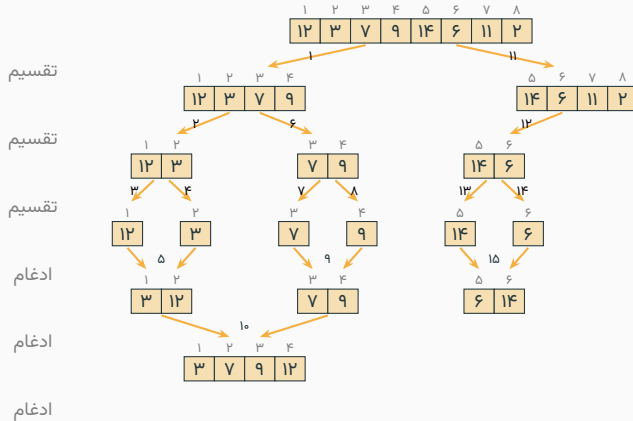
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT(A, 6, 6)

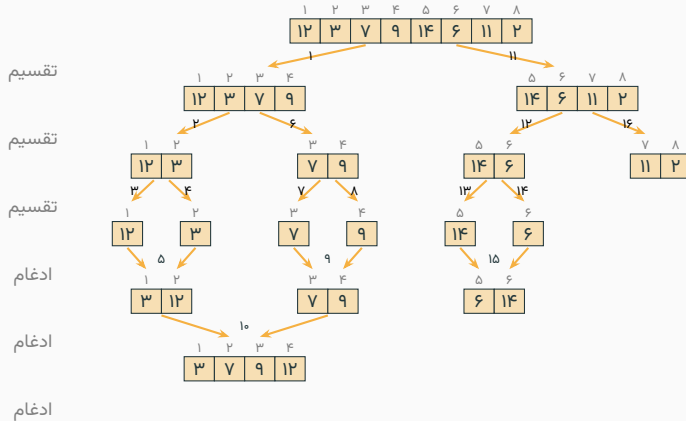
صفحه ۴۰ (بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) *CLRS, 4th ed.*



جاری فراخوانی

MERGE(A, 5, 5, 6)

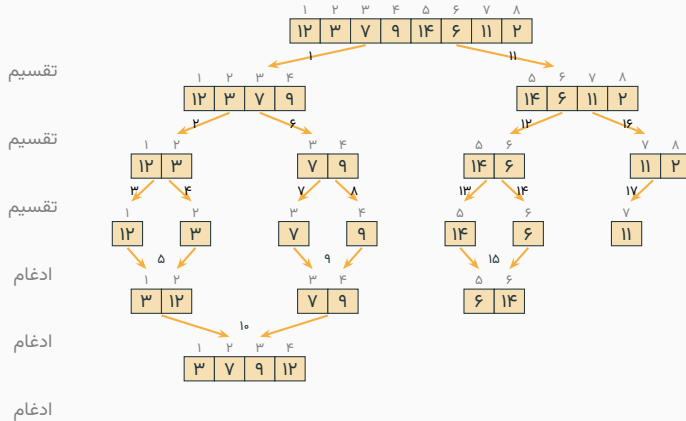
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT(A, 7, 8)

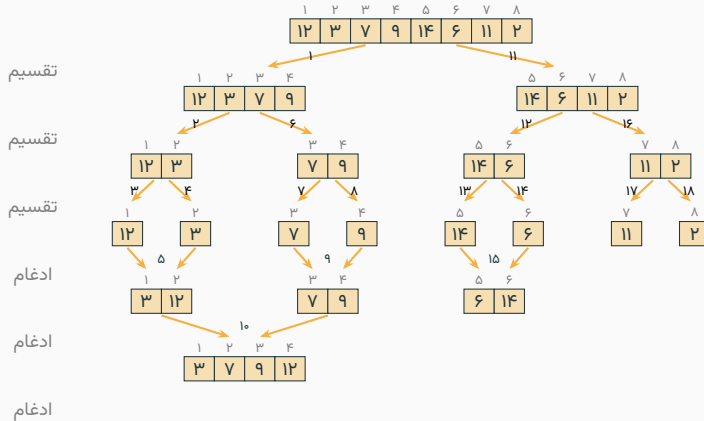
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT(A, 7, 7)

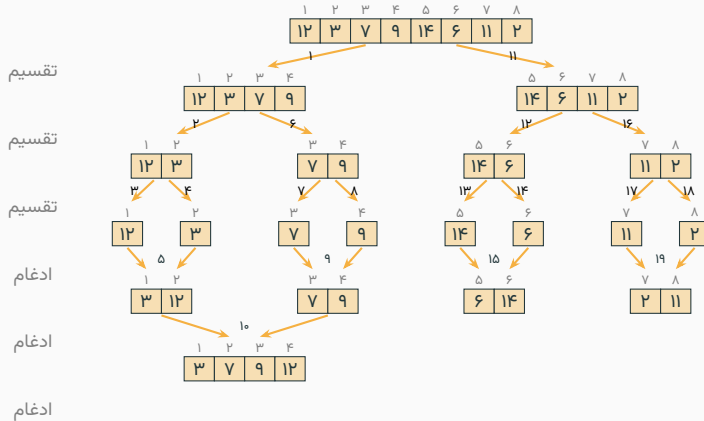
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE-SORT(A, 8, 8)

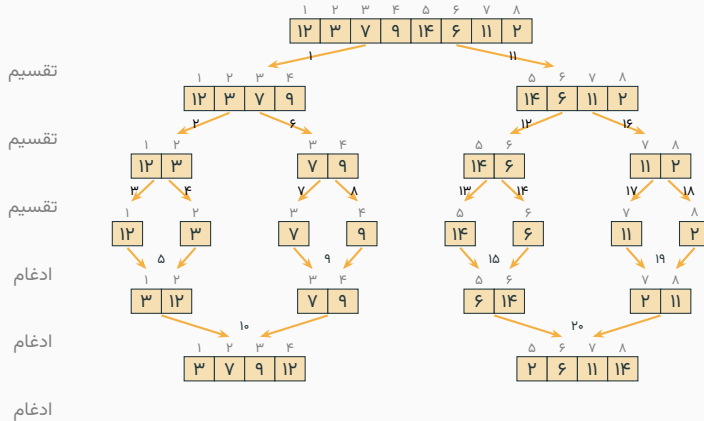
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE(A, 7, 7, 8)

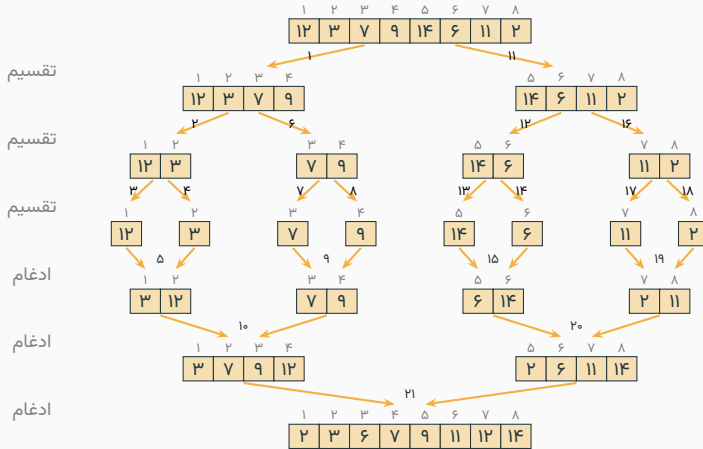
(بازترسیم شکل ۴.۲؛ برچسب‌های p ، q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE(A, 5, 6, 8)

(بازترسیم شکل ۴.۲؛ برچسب‌های p , q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*



جاری فراخوانی

MERGE(A, 1, 4, 8)

(بازترسیم شکل ۴.۲؛ برچسب‌های p , q و r حذف شده‌اند) صفحه ۴۰، *CLRS, 4th ed.*

«شکل ۴.۲ عملکرد این رویه را برای $n = ۸$ نشان می‌دهد و ترتیب گام‌های تقسیم و ادغام را هم نمایش می‌دهد. الگوریتم به صورت بازگشتی آرایه را تا زیرآرایه‌های ۱ عنصری تقسیم می‌کند. گام‌های ترکیب، جفت‌های زیرآرایه‌های ۱ عنصری را ادغام می‌کنند تا زیرآرایه‌های مرتب به طول ۲ بسازند، آن‌ها را ادغام می‌کنند تا زیرآرایه‌های مرتب به طول ۴ بسازند و آن‌ها را ادغام می‌کنند تا زیرآرایه‌ی مرتب نهایی به طول ۸ ساخته شود. اگر n توانی دقیق از ۲ نباشد، آنگاه برخی گام‌های تقسیم زیرآرایه‌هایی می‌سازند که طولشان ۱ واحد با هم فرق دارد. (برای مثال، وقتی زیرآرایه‌ای به طول ۷ تقسیم می‌شود، یک زیرآرایه طول ۴ و دیگری طول ۳ دارد.) صرف‌نظر از طول دو زیرآرایه‌ای که ادغام می‌شوند، زمان ادغام روی هم n عنصر $\Theta(n)$ است.»

CLRS, 4th ed.، صفحه ۳۸ و ۳۹ (ترجمه ماشینی)

رابطه‌ی بازگشتی

«وقتی الگوریتمی یک فراخوانی بازگشتی دارد، اغلب می‌توانید زمان اجرایش را با یک **معادله‌ی بازگشتی** یا **رابطه‌ی بازگشتی** توصیف کنید، که زمان اجرای کل روی مسئله‌ای به اندازه‌ی n را برحسب زمان اجرای همان الگوریتم روی ورودی‌های کوچک‌تر توصیف می‌کند. سپس می‌توانید از ابزارهای ریاضی برای حل رابطه‌ی بازگشتی و ارائه‌ی کران‌هایی برای کارایی الگوریتم استفاده کنید.»

CLRS, 4th ed.، صفحه ۳۹ (ترجمه ماشینی)

«رابطه‌ی بازگشتی برای زمان اجرای یک الگوریتم تقسیم و حل از سه گام روش پایه به دست می‌آید. همان‌طور که برای مرتب‌سازی درجی انجام دادیم، فرض کنید $T(n)$ زمان اجرای بدترین حالت روی مسئله‌ای به اندازه‌ی n باشد. اگر اندازه‌ی مسئله به اندازه‌ی کافی کوچک باشد، مثلاً $n < n_0$ برای ثابتی $n_0 > 0$ ، راه‌حل سرراست زمان ثابتی می‌گیرد که آن را $\Theta(1)$ می‌نویسیم. فرض کنید تقسیم مسئله a زیرمسئله بدهد که اندازه‌ی هر کدام n/b است، یعنی $1/b$ اندازه‌ی مسئله‌ی اصلی. برای مرتب‌سازی ادغامی، هم a و هم b برابر ۲ هستند، اما الگوریتم‌های تقسیم و حل دیگری خواهیم دید که در آن‌ها $a \neq b$. حل یک زیرمسئله به اندازه‌ی n/b زمان $T(n/b)$ می‌گیرد و بنابراین حل همه‌ی a زیرمسئله زمان $aT(n/b)$ می‌گیرد. اگر تقسیم مسئله به زیرمسئله‌ها زمان $D(n)$ و ترکیب جواب‌های زیرمسئله‌ها در جواب مسئله‌ی اصلی زمان $C(n)$ بگیرد، رابطه‌ی بازگشتی زیر را به دست می‌آوریم:»

$$T(n) = \begin{cases} \Theta(1), & n < n_0 \\ D(n) + aT(n/b) + C(n), & \text{در غیر این صورت} \end{cases}$$

CLRS, 4th ed.، صفحه ۳۹ و ۴۰ (ترجمه ماشینی)

«گاهی اندازه‌ی n/b در گام تقسیم عدد صحیح نیست. برای مثال، رویه‌ی MERGE-SORT مسئله‌ای به اندازه‌ی n را به زیرمسئله‌هایی به اندازه‌های $\lceil n/2 \rceil$ و $\lfloor n/2 \rfloor$ تقسیم می‌کند. چون تفاوت $\lceil n/2 \rceil$ و $\lfloor n/2 \rfloor$ حداکثر ۱ است، که برای n های بزرگ بسیار کمتر از اثر تقسیم n بر ۲ است، کمی چشم‌پوشی می‌کنیم و اندازه‌ی هر دو را فقط $n/2$ می‌نامیم. همان‌طور که فصل ۴ بحث خواهد کرد، این ساده‌سازی نادیده گرفتن کف‌ها و سقف‌ها به‌طور کلی بر مرتبه‌ی رشد جواب یک رابطه‌ی بازگشتی تقسیم و حل اثری نمی‌گذارد.»

«قرارداد دیگری که می‌پذیریم این است که بیان حالت‌های پایه‌ی رابطه‌ی بازگشتی را حذف کنیم، که آن را هم در فصل ۴ با جزئیات بیشتری بحث خواهیم کرد. دلیلش این است که حالت‌های پایه تقریباً همیشه $T(n) = \Theta(1)$ اگر $n < n_0$ برای ثابتی $n_0 > 0$ هستند. دلیل این است که زمان اجرای یک الگوریتم روی ورودی‌ای با اندازه‌ی ثابت، ثابت است. با پذیرفتن این قرارداد، از نوشتن مقدار زیادی مطلب اضافه خلاص می‌شویم.»

CLRS, 4th ed.، صفحه ۴۰ و ۴۱ (ترجمه ماشینی)

خط در MERGE-SORT	گام
خط ۳	$D(n) = \Theta(1)$
خط‌های ۴ و ۵	$2T(n/2)$
خط ۷	$C(n) = \Theta(n)$

«در این جا می‌بینید که رابطه‌ی بازگشتی برای $T(n)$ ، زمان اجرای بدترین حالت مرتب‌سازی ادغامی روی n عدد، چگونه نوشته می‌شود.

• **تقسیم:** گام تقسیم فقط وسط زیرآرایه را حساب می‌کند که زمان ثابتی می‌گیرد. بنابراین $D(n) = \Theta(1)$.

• **حل:** حل بازگشتی دو زیرمسئله، هر کدام به اندازه‌ی $n/2$ ، مقدار $2T(n/2)$ به زمان اجرا اضافه می‌کند (با نادیده گرفتن کف‌ها و سقف‌ها، همان‌طور که بحث کردیم).

• **ترکیب:** چون رویه‌ی MERGE روی یک زیرآرایه‌ی n عنصری زمان $\Theta(n)$ می‌گیرد، داریم $C(n) = \Theta(n)$.

(ترجمه ماشینی) *CLRS, 4th ed.*، صفحه ۴۱

«وقتی توابع $D(n)$ و $C(n)$ را برای تحلیل مرتب‌سازی ادغامی جمع می‌کنیم، تابعی را که $\Theta(n)$ است با تابعی که $\Theta(1)$ است جمع می‌کنیم. این مجموع تابعی خطی از n است. یعنی وقتی n بزرگ است تقریباً با n متناسب است، و بنابراین زمان‌های تقسیم و ترکیب مرتب‌سازی ادغامی روی هم $\Theta(n)$ هستند. افزودن $\Theta(n)$ به جمله‌ی $2T(n/2)$ از گام حل، رابطه‌ی بازگشتی زمان اجرای بدترین حالت $T(n)$ مرتب‌سازی ادغامی را به دست می‌دهد:

$$T(n) = 2T(n/2) + \Theta(n)$$

«

CLRS, 4th ed.، صفحه ۴۱ (ترجمه ماشینی)

«فصل ۴ “قضیه‌ی اصلی” را ارائه می‌کند که نشان می‌دهد $T(n) = \Theta(n \lg n)$. در مقایسه با مرتب‌سازی درجی، که زمان اجرای بدترین حالتش $\Theta(n^2)$ است، مرتب‌سازی ادغامی یک ضریب n را با یک ضریب $n \lg n$ معاوضه می‌کند. چون تابع لگاریتم کندتر از هر تابع خطی رشد می‌کند، این معامله‌ی خوبی است. برای ورودی‌های به‌اندازه‌ی کافی بزرگ، مرتب‌سازی ادغامی با زمان اجرای بدترین حالت $\Theta(n \lg n)$ از مرتب‌سازی درجی، که زمان اجرای بدترین حالتش $\Theta(n^2)$ است، بهتر عمل می‌کند.»

CLRS, 4th ed.، صفحه ۴۱ (ترجمه ماشینی)

درخت بازگشت

«اما برای اینکه به‌طور شهودی بفهمیم چرا جواب رابطه‌ی بازگشتی (۳.۲) برابر $T(n) = \Theta(n \lg n)$ است، به قضیه‌ی اصلی نیازی نداریم. برای سادگی، فرض کنید n توانی دقیق از ۲ است و حالت پایه‌ی ضمنی $n = 1$ است. آنگاه رابطه‌ی بازگشتی (۳.۲) در اصل چنین است:

$$T(n) = \begin{cases} c_1, & n = 1 \\ 2T(n/2) + c_2 n, & n > 1 \end{cases} \quad (۲/۴)$$

که در آن ثابت $c_1 > 0$ زمان لازم برای حل مسئله‌ای به اندازه‌ی ۱ را نشان می‌دهد و $c_2 > 0$ زمان به ازای هر عنصر آرایه در گام‌های تقسیم و ترکیب است.»

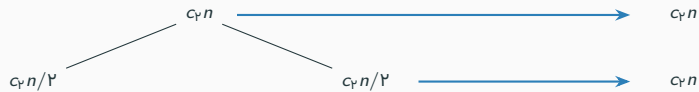
CLRS, 4th ed.، صفحه ۴۲ (ترجمه ماشینی)

$O(n)$

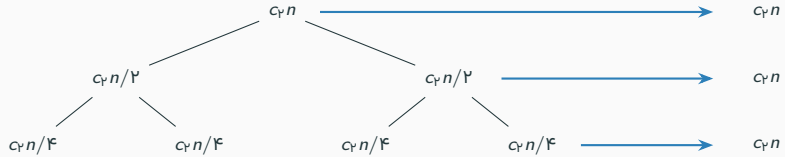
d) *CLRS, 4th ed.* صفحه ۴۳ (بازترسیم شکل ۵.۲، بخش d).

$$c_2 n \longrightarrow c_2 n$$

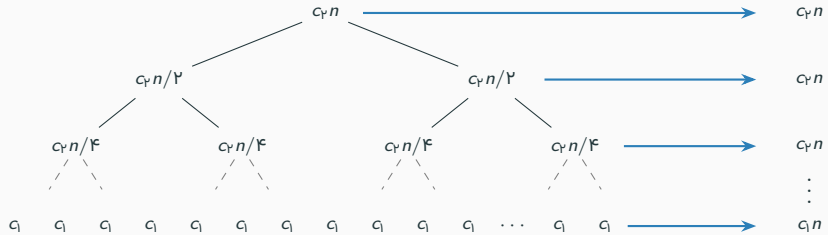
d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲)، *CLRS*, 4th ed.



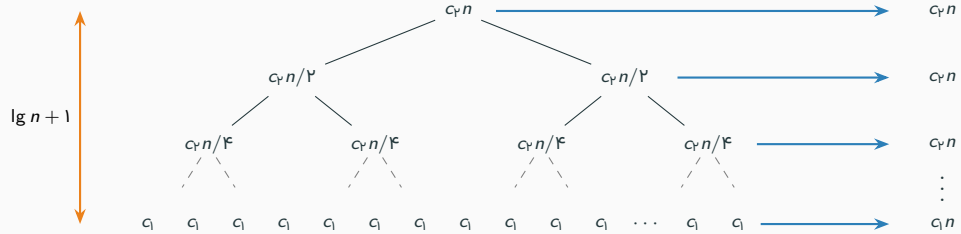
d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲، *CLRS*, 4th ed.



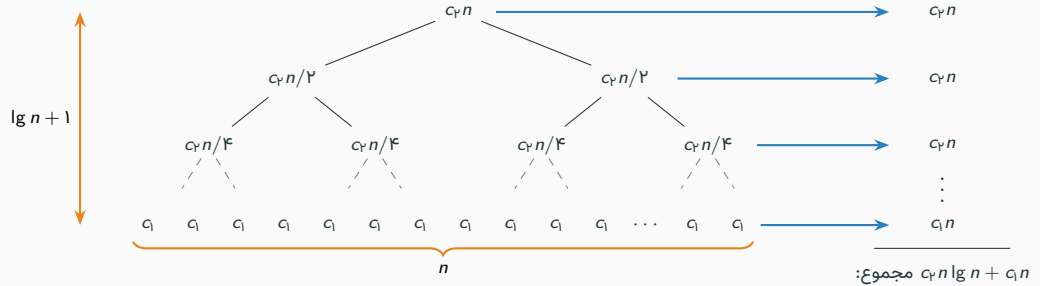
d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲)، *CLRS*, 4th ed.



d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲، *CLRS*, 4th ed.



d) بخش ۵.۲، شکل ۴۳ (بازترسیم شکل ۵.۲، بخش ۵.۲)، *CLRS*, 4th ed.



d) بخش ۵.۲، شکل ۴۳ (بازترسیم *CLRS*, 4th ed. صفحه ۴۳)

«سپس هزینه‌ها را در هر سطح درخت جمع کنید. سطح بالا هزینه‌ی کل $c_2 n$ دارد، سطح پایینی بعدی هزینه‌ی کل $c_2(n/2) + c_2(n/2) = c_2 n$ ، سطح پس از آن هزینه‌ی کل $c_2(n/4) + c_2(n/4) + c_2(n/4) + c_2(n/4) = c_2 n$ ، و به همین ترتیب. هر سطح دو برابر سطح بالایی‌اش گره دارد، اما هر گره فقط نصف هزینه‌ی یک گره از سطح بالایی را سهم دارد. از یک سطح به سطح بعد، دو برابر شدن و نصف شدن یکدیگر را خنثی می‌کنند، به‌طوری که هزینه در همه‌ی سطح‌ها یکسان است: $c_2 n$. به‌طور کلی، سطحی که i سطح پایین‌تر از بالاست 2^i گره دارد که هر کدام هزینه‌ی $c_2(n/2^i)$ را سهم دارند، به‌طوری که سطح i ام پایین‌تر از بالا هزینه‌ی کل $c_2(n/2^i) \cdot 2^i = c_2 n$ دارد. پایین‌ترین سطح n گره دارد که هر کدام هزینه‌ی c_1 را سهم دارند، که هزینه‌ی کل آن $c_1 n$ است.»

CLRS, 4th ed.، صفحه ۴۲ (ترجمه ماشینی)

«تعداد کل سطح‌های درخت بازگشت در شکل ۵.۲ برابر $\lg n + 1$ است، که در آن n تعداد برگ‌هاست که با اندازه‌ی ورودی متناظر است. یک استدلال استقرایی غیررسمی این ادعا را توجیه می‌کند. حالت پایه وقتی رخ می‌دهد که $n = 1$ ، که در آن حالت درخت فقط ۱ سطح دارد. چون $\lg 1 = 0$ ، داریم که $\lg n + 1$ تعداد درست سطح‌ها را می‌دهد. اکنون به‌عنوان فرض استقرا فرض کنید تعداد سطح‌های یک درخت بازگشت با 2^i برگ برابر $i + 1 = \lg 2^i + 1$ است (چون برای هر مقدار i داریم $\lg 2^i = i$). چون فرض می‌کنیم اندازه‌ی ورودی توانی دقیق از ۲ است، اندازه‌ی ورودی بعدی که باید در نظر بگیریم 2^{i+1} است. درختی با $n = 2^{i+1}$ برگ ۱ سطح بیشتر از درختی با 2^i برگ دارد، و بنابراین تعداد کل سطح‌ها $1 + \lg 2^{i+1} = (i + 1) + 1$ است.»

CLRS, 4th ed.، صفحه ۴۲ و ۴۴ (ترجمه ماشینی)

«برای محاسبه‌ی هزینه‌ی کلی که رابطه‌ی بازگشتی (۴.۲) نشان می‌دهد، کافی است هزینه‌های همه‌ی سطرها را جمع کنید. درخت بازگشت $\lg n + 1$ سطح دارد. هزینه‌ی هر یک از سطرها بالای برگ‌ها $c_2 n$ است و هزینه‌ی سطح برگ‌ها $c_1 n$ ، که هزینه‌ی کل آن $c_2 n \lg n + c_1 n = \Theta(n \lg n)$ است.»

CLRS, 4th ed.، صفحه ۴۴ (ترجمه ماشینی)

$$T(n) = \underbrace{c_2 n + c_2 n + \dots + c_2 n}_{\lg n} + \underbrace{c_1 n}_{\text{برگ‌ها}} = c_2 n \lg n + c_1 n = \Theta(n \lg n)$$

منابع

Cormen, Leiserson, Rivest, Stein — *Introduction to Algorithms* (CLRS), 4th Edition, MIT Press, 2022 •